НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені Ігоря Сікорського»
ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ

Кафедра системного програмування та спеціалізованих комп'ютерних систем

Лабораторна робота №3

з дисципліни Web-дизайн

на тему: «Розробка Web-додатка засобами Javascript/VueJS»

Виконав:
студент III курсу
групи КВ-34
Чижов В.Ю

Перевірив:

Київ – 2026

Мета: ознайомитись із засобами фреймворка VueJS та навчитись створювати асинхронні запити до Web-сервера.

Загальне завдання: адаптувати програмний код ЛР№2 до вимог фреймворка VueJS та забезпечити завантаження необхідних даних з Web-сервера.

Інструменти розробки: Javascript, VueJS, NodeJS, ExpressJS, SQLite.

Репозиторій Git: https://github.com/mehnn/lab3_web
Github Pages / Railway: https://lab3web-production.up.railway.app/
Telegram: @chizhovl

Додаток

Назва: CHIIIIKS — Словник англійської мови.

Опис:

CHIIIIKS — це навчальний Web-додаток для вивчення англійської лексики. Додаток дозволяє переглядати слова за рівнями (A1, A2, B1) і темами, вивчати їх у режимі карток та перевіряти знання у режимі вікторини. Дані про прогрес зберігаються на сервері у базі даних SQLite. Автентифікація реалізована через сесії на базі in-memory сховища сервера ExpressJS.

Призначення та основні можливості:

- реєстрація та вхід в акаунт;
- перегляд словника за рівнями A1 / A2 / B1 та темами (Everyday, Food, Travel, Work);

- вивчення карток: перегортання EN/UA, позначення слова як "Знаю" / "Не знаю";
- відображення прогресу у вигляді прогрес-бару (% вивчених слів);
- режим вікторини: вибір правильного перекладу з 4 варіантів;
- фільтр "тільки невивчені" у вікторині;
- облік статистики квізу (кількість правильних/загальних відповідей);
- перегляд профілю з персональною статистикою;
- можливість скинути прогрес;
- адаптивний інтерфейс (Bootstrap 5, mobile-first).

Перелік VueJS-компонентів додатка та схема їх взаємодії

У реалізованому додатку використовується кореневий VueJS-компонент App, який монтується в контейнер #app та керує маршрутизацією (SPA) за допомогою реактивної змінної page. У попередній версії функції навігації реалізовувались окремими JS-модулями без фреймворку; у новій версії кожна сторінка стала окремим компонентом:

Компонент App (vue/App.js):

Кореневий компонент. Зберігає стан авторизації (user), поточну сторінку (page), виконує перевірку сесії при старті через apiMe(). Монтує дочірні компоненти LoginPage, StudyPage, QuizPage, ProfilePage та відображає навігаційний бар і футер.

Компонент LoginPage (vue/components/LoginPage.js):

Компонент входу та реєстрації. Містить перемикач між формами, поля для введення email/пароля/імені/статі/дати народження. Виконує запити apiLogin() та apiRegister() і після успіху генерує подію logged-in.

Компонент StudyPage (vue/components/StudyPage.js):

Компонент режиму вивчення. Реалізує картки (flip EN/UA), навігацію між картками, позначення слів як вивчених/невивчених через apiSetKnown(). Відображає прогрес-бар та таблицю всіх слів поточної теми з їхніми статусами.

Компонент QuizPage (vue/components/QuizPage.js):

Компонент режиму вікторини. Формує питання методом makeOptions() — правильна відповідь + 3 випадкових варіанти. Фіксує результат через apiRecordQuiz(). Підтримує фільтр "тільки невивчені" та кнопку перезапуску.

Компонент ProfilePage (vue/components/ProfilePage.js):

Компонент профілю. Отримує дані з apiGetProgress() та відображає: кількість вивчених слів, загальний прогрес (%), точність квізу. Має кнопку виходу (apiLogout()).

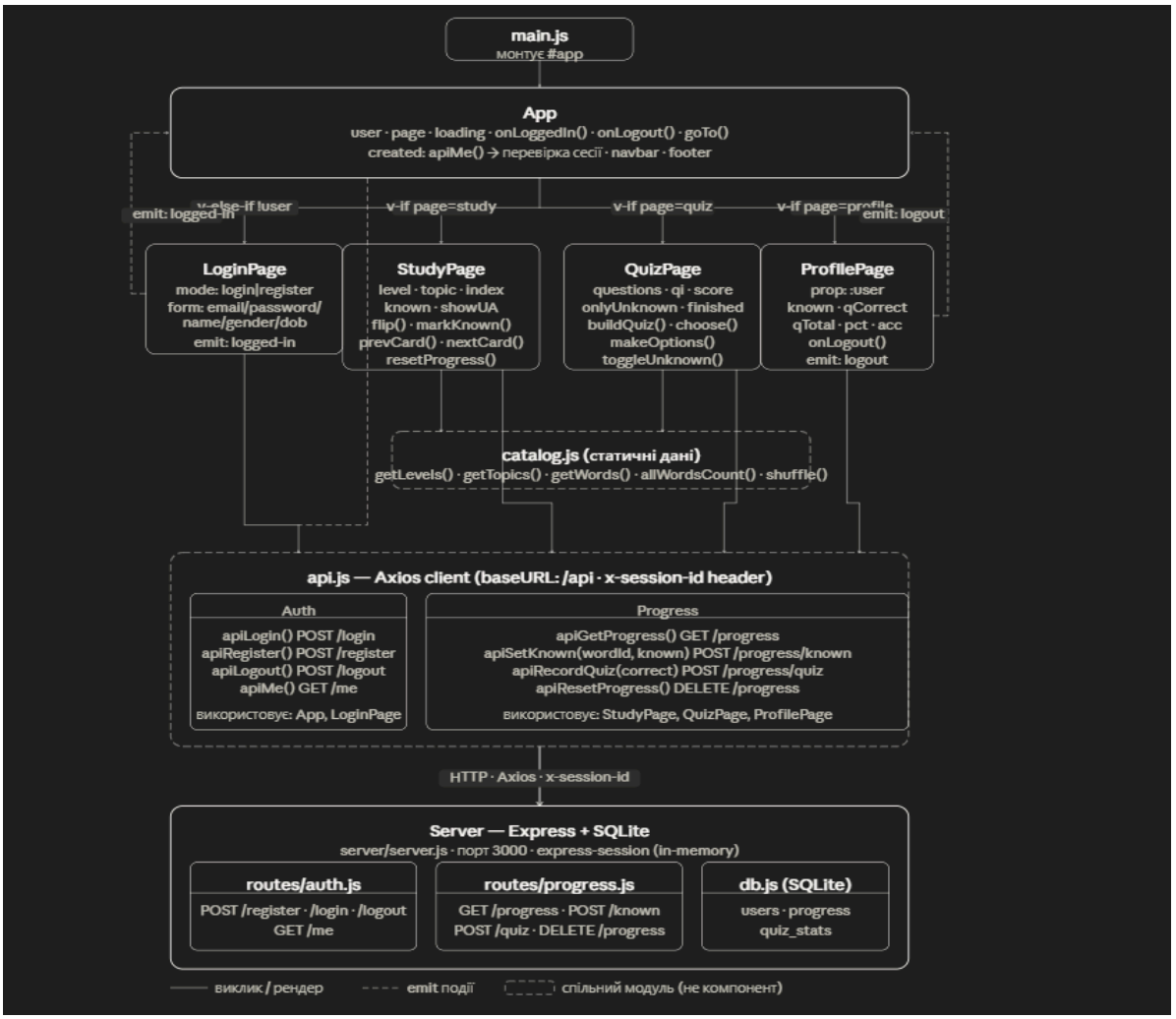
Структура проекту

```
vue_project/
│
├── server/
│   ├── server.js          — Express-сервер, сесії, статика
│   ├── db.js              — SQLite: ініціалізація таблиць, run/get/all
│   └── routes/
│       ├── auth.js        — /api/register, /api/login, /api/logout, /api/me
│       └── progress.js    — /api/progress (GET/POST/DELETE)
│
├── public/
│   ├── index.html         — SPA-оболонка, підключення Bootstrap/Vue/Axios
│   ├── style.css          — глобальні стилі (CSS-змінні, glass, kpi, badge...)
│   └── vue/
│       ├── main.js        — createApp(App).mount('#app')
│       ├── api.js         — Axios-клієнт, interceptors для x-session-id
│       ├── catalog.js     — статичний словник A1/A2/B1 + утиліти
getLevels/getTopics/getWords
│       ├── App.js         — кореневий компонент
│       └── components/
│
```

```
│           ├── LoginPage.js
│           ├── StudyPage.js
│           ├── QuizPage.js
│           └── ProfilePage.js
│
└── package.json
```

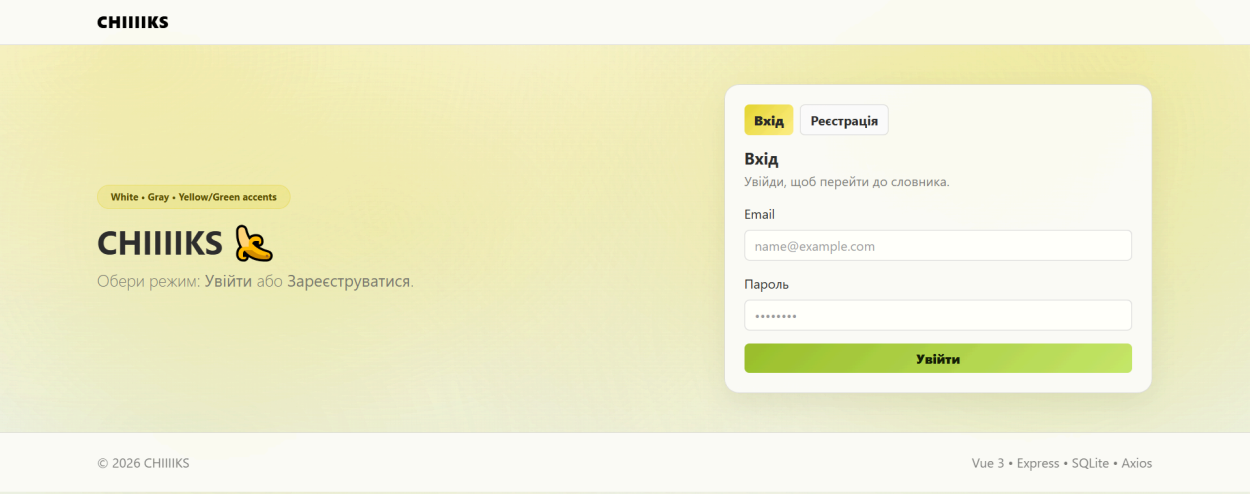
Схема взаємодії VueJS-компонентів

Схему взаємодії компонентів додатка CHIIIIKS наведено нижче. Кореневий компонент App містить стан user та page. Дочірні компоненти отримують дані через props та повертають події через $emit. Всі звернення до сервера інкапсульовані у модулі api.js (Axios + interceptors для сесійного заголовка x-session-id).

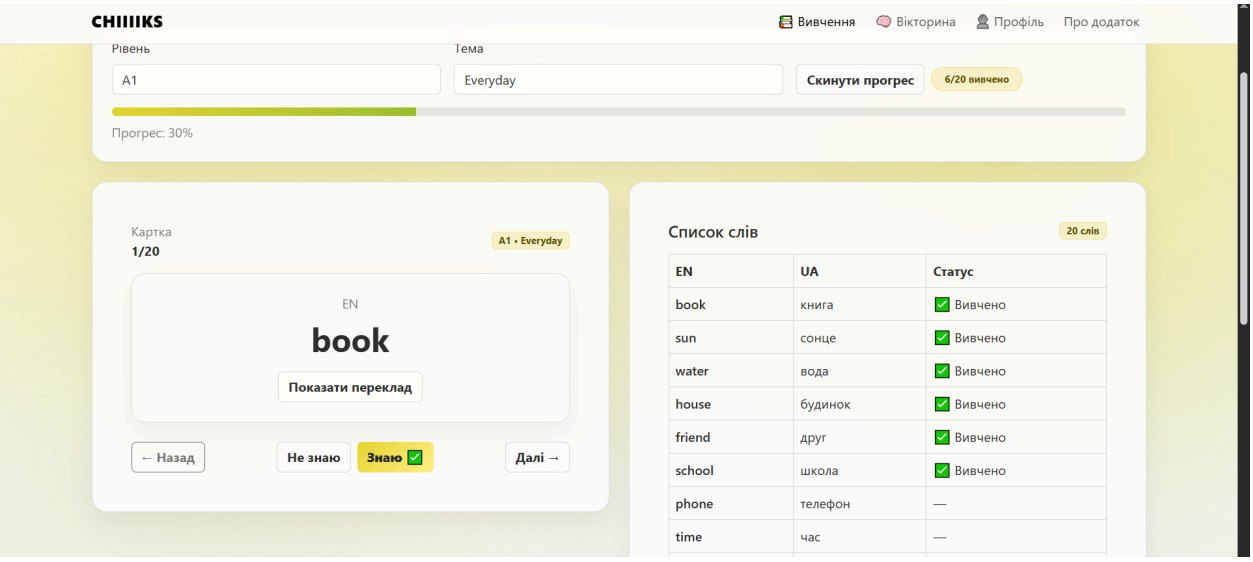


Скріншоти всіх сторінок Web-додатка

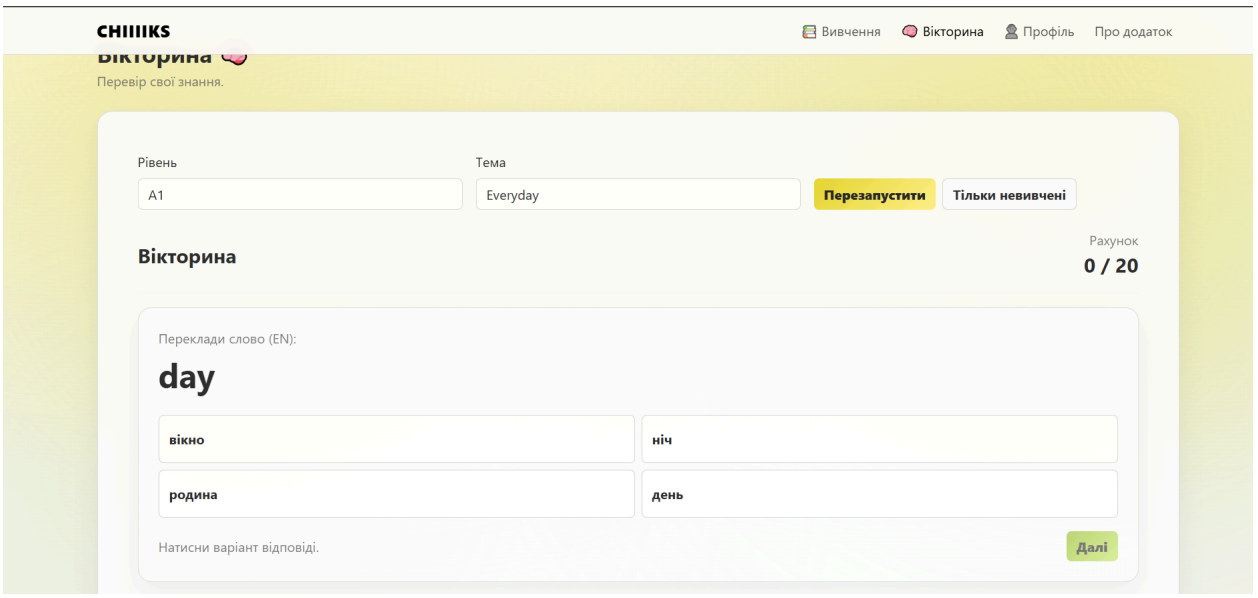
Скріншоти всіх сторінок Web-додатка CHIIIIKS у десктопній версії наведено на рисунках 1–5.



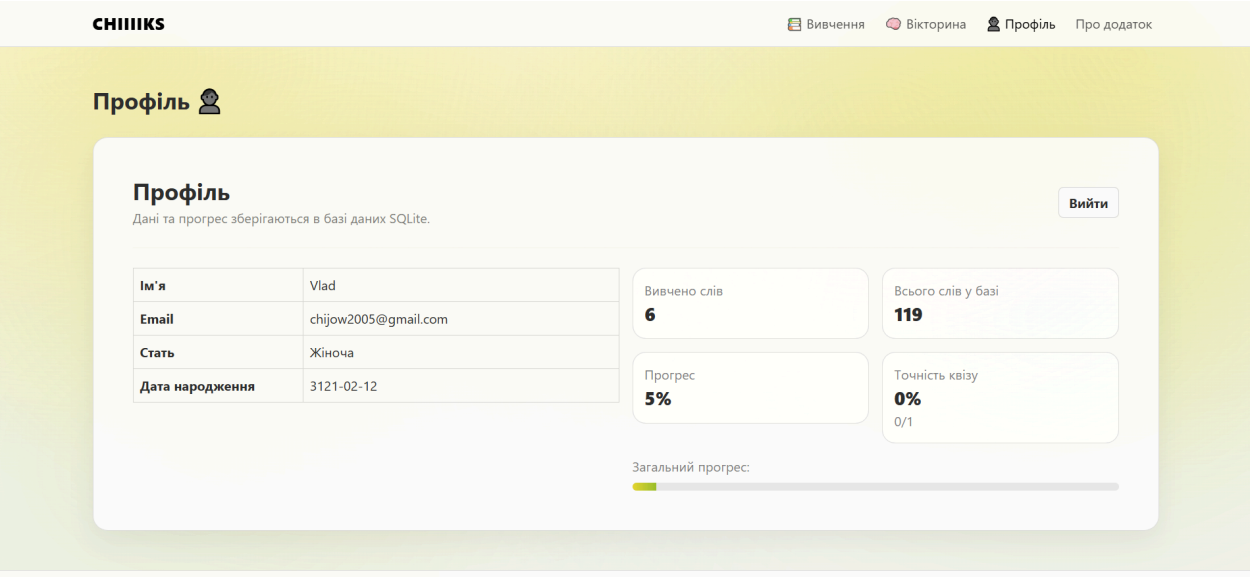
[Рис. 1 – Сторінка входу та реєстрації (LoginPage)]



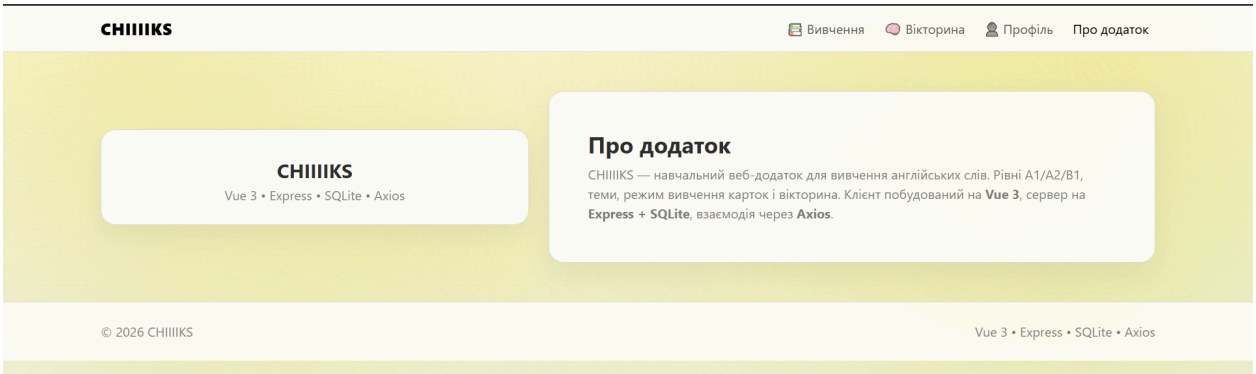
[Рис. 2 – Сторінка вивчення карток (StudyPage)]



[Рис. 3 – Режим вікторини (QuizPage)]



[Рис. 4 – Профіль користувача (ProfilePage)]



[Рис. 5 – Сторінка "Про додаток"]

Код компонентів

package.json

Файл конфігурації NodeJS-проєкту. Містить назву, версію, залежності (express, cors, sql.js) та сценарії запуску (start, dev через nodemon).

```
{
  "name": "chiiiiks-vue",
```

```
  "version": "1.0.0",
  "description": "CHIIIIKS — Vue 3 + Express + SQLite",
  "main": "server/server.js",
  "scripts": {
    "start": "node server/server.js",
    "dev": "nodemon server/server.js"
  },
  "dependencies": {
    "express": "^4.18.3",
    "cors": "^2.8.5",
    "sql.js": "^1.10.3"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

server/db.js

Модуль роботи з базою даних SQLite. Відповідає за підключення до бази, створення таблиць users, known_words, quiz_stats та надає обгортки run(), get(), all() для зручної роботи через Promise.

```
const path    = require('path');
const sqlite3 = require('sqlite3').verbose();

const dbPath = path.join(__dirname, 'app.db');
const db     = new sqlite3.Database(dbPath);

function run(sql, params = []) {
  return new Promise((resolve, reject) => {
    db.run(sql, params, function(err) {
      if (err) reject(err); else resolve(this);
    });
  });
}

function get(sql, params = []) {
  return new Promise((resolve, reject) => {
    db.get(sql, params, (err, row) => {
      if (err) reject(err); else resolve(row);
    });
  });
}

function all(sql, params = []) {
  return new Promise((resolve, reject) => {
    db.all(sql, params, (err, rows) => {
      if (err) reject(err); else resolve(rows);
    });
  });
}
```

```
async function initDb() {
  await run(`CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL, email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL, gender TEXT DEFAULT 'Інше',
    dob TEXT DEFAULT '', created_at TEXT NOT NULL
  )`);
  await run(`CREATE TABLE IF NOT EXISTS known_words (
    email TEXT NOT NULL, word_id TEXT NOT NULL,
    PRIMARY KEY (email, word_id)
  )`);
  await run(`CREATE TABLE IF NOT EXISTS quiz_stats (
    email TEXT PRIMARY KEY, correct INTEGER DEFAULT 0, total INTEGER DEFAULT 0
  )`);
}

module.exports = { db, run, get, all, initDb };
```

server/server.js

Основний файл серверної частини на ExpressJS. Ініціалізує базу даних, підключає middleware (cors, json, in-memory сесії через Map), роздає статику з public/ та підключає маршрути /api/auth та /api/progress.

```
const express  = require('express');
const path     = require('path');
const cors     = require('cors');
const { initDb } = require('./db');

const app  = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(cors({ exposedHeaders: ['x-session-id'] }));

// In-memory сесії
const sessions = new Map();
app.use((req, res, next) => {
  let sid = req.headers['x-session-id'];
  if (!sid || !sessions.has(sid)) {
    sid = Math.random().toString(36).slice(2) + Date.now();
    sessions.set(sid, {});
  }
  req.session = sessions.get(sid);
  res.setHeader('x-session-id', sid);
  next();
});

app.use(express.static(path.join(__dirname, '..', 'public')));
```

```
initDb().then(() => {
  app.use('/api', require('./routes/auth'));
  app.use('/api/progress', require('./routes/progress'));
  app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname, '..', 'public', 'index.html'));
  });
  app.listen(PORT, () => console.log(`Running → http://localhost:${PORT}`));
}).catch(err => { console.error(err); process.exit(1); });
```

server/routes/auth.js

Маршрути авторизації. POST /api/register — реєстрація нового користувача; POST /api/login — вхід з перевіркою пароля та збереженням email у сесію; POST /api/logout — очищення сесії; GET /api/me — повернення даних поточного користувача.

```
// POST /api/register
router.post('/register', (req, res) => {
  const { name, email, gender, dob, password } = req.body;
  if (!name || !email || !password)
    return res.status(400).json({ error: "Заповни ім'я, email та пароль" });
  run('INSERT INTO users (name,email,gender,dob,password) VALUES (?,?,?,?,?)',
      name, email, gender??'', dob??'', password);
  res.json({ ok: true });
});

// POST /api/login
router.post('/login', (req, res) => {
  const { email, password } = req.body;
  const user = get(
    'SELECT id,name,email,gender,dob FROM users WHERE email=? AND password=?',
    email, password);
  if (!user) return res.status(401).json({ error: 'Невірний email або пароль'
});
  req.session.email = user.email;
  res.json({ ok: true, user });
});
```

server/routes/progress.js

Маршрути прогресу. GET /api/progress — повернення об'єкта known (вивчені слова) та статистики квізу; POST /api/progress/known — позначення слова як вивченого чи невивченого; POST /api/progress/quiz — запис результату відповіді у вікторині; DELETE /api/progress — повне скидання прогресу поточного користувача.

```
// GET /api/progress
router.get('/', async (req, res) => {
  const email = req.session.email;
  const rows  = await all('SELECT word_id FROM known_words WHERE email=?',
email);
  const known = {};
  rows.forEach(r => { known[r.word_id] = true; });
  const stats = await get('SELECT correct,total FROM quiz_stats WHERE email=?',
email)
    ?? { correct: 0, total: 0 };
  res.json({ known, quiz: stats });
});

// POST /api/progress/known
router.post('/known', async (req, res) => {
  const { wordId, known } = req.body;
  if (known) {
    await run('INSERT OR IGNORE INTO known_words (email,word_id) VALUES (?,?)',
              req.session.email, wordId);
  } else {
    await run('DELETE FROM known_words WHERE email=? AND word_id=?',
              req.session.email, wordId);
  }
  res.json({ ok: true });
});
```

vue/api.js

Модуль HTTP-клієнта на базі Axios. Зберігає sessionId у localStorage та передає його в заголовку x-session-id через request-interceptor. Response-interceptor оновлює sid при зміні. Експортує функції: apiRegister, apiLogin, apiLogout, apiMe, apiGetProgress, apiSetKnown, apiRecordQuiz, apiResetProgress.

```
let _sid = localStorage.getItem('sid') || null;
const http = axios.create({ baseURL: '/api' });

http.interceptors.request.use(cfg => {
  if (_sid) cfg.headers['x-session-id'] = _sid;
  return cfg;
});

http.interceptors.response.use(res => {
  const sid = res.headers['x-session-id'];
  if (sid && sid !== _sid) {
    _sid = sid;
    localStorage.setItem('sid', sid);
  }
  return res;
```

```
});

const apiLogin  = data => http.post('/login', data);
const apiLogout = ()   => http.post('/logout')
  .then(() => { localStorage.removeItem('sid'); _sid = null; });
const apiMe     = ()   => http.get('/me').then(r => r.data.user);
const apiGetProgress = () => http.get('/progress').then(r => r.data);
const apiSetKnown = (id, known) => http.post('/progress/known', { wordId:id,
known });
const apiRecordQuiz = correct => http.post('/progress/quiz', { correct });
const apiResetProgress = () => http.delete('/progress');
```

vue/App.js

Кореневий Vue-компонент. Зберігає стан user (об'єкт або null) та page (рядок). У хуку created() виконує apiMe() для відновлення сесії. Методи onLoggedIn(), onLogout(), goTo() змінюють стан та перемикають сторінки. Шаблон містить навбар, умовний рендеринг LoginPage/StudyPage/QuizPage/ProfilePage та статичну сторінку «Про додаток».

vue/components/StudyPage.js

Компонент карток. Computed-властивості levels, topics, words, card, knownCount, pct автоматично перераховуються при зміні level/topic/known. Методи flip(), prevCard(), nextCard() керують навігацією, markKnown(isKnown) синхронізує локальний стан з сервером через apiSetKnown().

vue/components/QuizPage.js

Компонент вікторини. Метод buildQuiz() формує масив питань: для кожного слова функція makeOptions() бере 3 випадкових неправильних варіанти з поточного словника. choose(opt) блокує повторне натискання (locked), перевіряє правильність та викликає apiRecordQuiz(). next() переходить до наступного питання або встановлює finished = true.

vue/components/LoginPage.js

Компонент авторизації. Перемикач mode (login/register) показує відповідну форму. onLogin() викликає apiLogin() і при успіху генерує подію logged-in з об'єктом user. onRegister() спочатку реєструє, потім автоматично логінить користувача.

vue/components/ProfilePage.js

Компонент профілю. Отримує props user від App. У created() завантажує прогрес через apiGetProgress() та обчислює pct (% словника) і acc (% точності квізу). Кнопка «Вийти» викликає apiLogout() та генерує подію logout.

Висновок

У ході виконання лабораторної роботи було виконано адаптацію Web-додатка CHIIIIKS до вимог фреймворка VueJS та архітектури клієнт–сервер. Якщо у попередній реалізації інтерфейс будувався на чистому JavaScript без фреймворку та зберігав дані у localStorage, то в новій версії стан інтерфейсу керується Vue 3, а прогрес і дані користувача зберігаються на сервері у SQLite через REST API на ExpressJS.

У результаті реалізовано повноцінний SPA-додаток: кореневий компонент App управляє маршрутизацією та авторизацією, дочірні компоненти StudyPage, QuizPage та ProfilePage отримують дані через props та API, модуль api.js централізує всі HTTP-запити з підтримкою сесій через заголовок x-session-id. Структура застосунку стала більш цілісною, масштабованою та відповідає реальним підходам до розробки сучасних Web-додатків.